MODULE 16 · DOMAIN 2 - ML MODEL AND FM DEVELOPMENT

Selecting and customising FMs

Prompts, inference parameters, the Converse API, tools, and when to train.

MODULE 16 of 43 DOMAIN 2 - Model Dev WEIGHT 24%

admin@sthithapragna.com

Most problems attributed to the **model** are actually problems with the **prompt or the parameters**.

Five ways to change what a model produces

TECHNIQUE	COST AND EFFECT
Inference parameters	Free. Changes randomness, length and stopping
Prompt engineering	Nearly free. Changes instructions, examples and framing
Tool use	Cheap. Lets the model call your systems for facts and actions
Retrieval (RAG)	Moderate. Grounds answers in your documents. Modules 09, 10, 26
Fine-tuning or distillation	Expensive. Changes the weights. Module 11

ON THE EXAM

Work down this list in order. The bottom row is the one everybody reaches for first, and the one that is right least often.

System prompts

- A **separate top-level field** in Converse, not another user message.
- It carries the durable instructions: role, tone, constraints, output rules.
- Models weight it differently from user turns, so rules stated here hold better across a conversation.
- It is also the natural place for the **cacheable prefix**, since it rarely changes.
- Putting standing rules in every user message wastes tokens and works less reliably.

> ON THE EXAM
>
> Anything true for every request belongs in the system prompt. Anything specific to this request belongs in the user message.

What actually improves a prompt

CHANGE	WHY IT WORKS
Be specific about the output	Vague instructions get varied output. Name the format exactly
Give two to five examples	Few-shot teaches format faster than any description of it
State what NOT to do	Explicit prohibitions work better than hoping it will not
Give the model a role	Framing shifts vocabulary and depth measurably
Put instructions before the data	Long documents can bury an instruction placed after them
Ask for reasoning first	Chain of thought helps on multi-step problems

ON THE EXAM

Prompt engineering has a bad reputation because it sounds unserious. It is the highest return-per-hour work available here.

Chain of thought and reasoning models

- **Chain of thought** asks the model to work through steps before giving an answer.
- It measurably improves multi-step arithmetic, logic and planning tasks.
- It costs output tokens, and output tokens are the expensive kind.
- **Reasoning models** do this natively, with an extended thinking mode built in.
- **Do not set temperature, topP or topK with extended thinking.** Those parameters are incompatible with it.

> ON THE EXAM
>
> The last bullet is the current detail: sampling parameters and reasoning modes do not mix, and setting them is a configuration error.

Four inference parameters control most of what people complain about.

The four parameters in inferenceConfig

PARAMETER	WHAT IT CONTROLS
`temperature`	Randomness. Low is focused and repeatable, high is varied and creative
`topP`	Nucleus sampling. Considers only tokens within the top probability mass
`maxTokens`	The maximum output length before generation stops
`stopSequences`	Custom text that causes the model to stop generating

ON THE EXAM

These four are the ONLY parameters in inferenceConfig on the Converse API. Everything else is model-specific.

Temperature zero is not a guarantee

- Temperature at zero makes output **near**-deterministic, not provably deterministic.
- Floating-point behaviour and serving-side changes can still produce variation.
- So **validate output rather than assuming it**, even on an extraction task at temperature zero.
- A model version change will alter outputs regardless of temperature.
- For a decision that must be identical every time, a rule or a classical model is the safer answer.

ON THE EXAM

Module 13's point returns: determinism is a genuine reason to prefer a classical model for a regulated decision.

Temperature, and what it actually does

- **Near zero** makes the model pick the most likely token almost every time. Near deterministic.
- Use it for **extraction, classification and structured output**, where you want the same answer twice.
- **Higher** flattens the distribution, so less likely tokens get chosen more often.
- Use it for brainstorming, creative writing and generating varied options.
- It does not make the model more or less **correct**. It makes it more or less **varied**.

> ON THE EXAM
>
> Inconsistent output on a repeated task is a temperature problem far more often than a model problem.

temperature or topP?

TEMPERATURE

Reshapes the whole distribution

- Scales the probabilities before sampling.
- Low values concentrate on the likeliest tokens.
- The usual first dial to reach for.
- Works on every model.

TOPP

Truncates the distribution

- Keeps only tokens within a probability mass.
- Cuts off the long tail of unlikely tokens.
- Adapts to how confident the model is.
- Also in the common inferenceConfig set.

THE TIEBREAKER

Tuning both at once makes the effect hard to reason about. Pick one, usually temperature, and leave the other at its default.

Why topK does not work in inferenceConfig

- **topK is not in the common set.** Putting it in inferenceConfig has no effect.
- It must be passed through **additionalModelRequestFields**, because it is model-specific.
- topK keeps the k most likely tokens, regardless of their probability mass.
- That field is the general escape hatch for **any parameter outside the common four**.
- Reasoning and extended-thinking modes are incompatible with temperature, topP and topK. Do not set them.

> ON THE EXAM
>
> The symptom is silence: no error, no effect. A parameter in the wrong place is simply ignored.

maxTokens and stopSequences

- **maxTokens** caps output length. Generation simply stops when it is reached.
- Hitting the cap produces a **truncated answer, not an error**. The response looks finished and is not.
- It is a cost control as well: output tokens are usually priced higher than input tokens.
- **stopSequences** halt generation when specific text appears. Useful for structured output.
- Limits vary by model. DeepSeek R1, for example, allows up to 32,768 output tokens.

> ON THE EXAM
>
> An answer that ends mid-sentence is almost always maxTokens, and almost never the model failing.

The Converse API

- **One consistent interface** across every model that supports messages. Write once, swap models.
- Takes **messages**, a **system** prompt, **inferenceConfig**, and optionally tools and guardrails.
- **modelId accepts four things:** a base model, an inference profile, a provisioned model ARN, or a custom model ARN.
- **It does not support imported models** from Custom Model Import.
- Bedrock does not store the text, images or documents you send as content.

> ON THE EXAM
>
> The portability is the point. Changing model becomes a modelId change rather than rewriting request and response handling.

Images and documents in a request

- A message's content array can carry **text, images and documents**, not just text.
- So you can ask questions about a diagram or a PDF without a separate extraction step.
- Support varies by model, so check before designing around it.
- Images consume a substantial number of input tokens. Cost rises faster than it feels like it should.
- For bulk document processing, module 10's Bedrock Data Automation is usually the better route.

> ON THE EXAM
>
> For a handful of documents, send them in the prompt. For thousands, parse them properly first.

Converse is stateless

- **There is no server-side session.** Bedrock does not remember your conversation.
- Your application maintains the **messages** array and resends the **full history** every request.
- So conversation length drives token cost directly. Every turn re-pays for every earlier turn.
- Long conversations eventually exceed the context window and must be trimmed or summarised.
- **Prompt caching** from module 12 is the usual mitigation for a repeated prefix.

> ON THE EXAM
>
> 'The chatbot forgot what I said' is not a model failure. It means the application did not resend the history.

What to do when the conversation gets long

- Every model has a **context window**. History plus retrieved chunks plus output must all fit.
- **Sliding window:** keep the last N turns and drop the oldest. Simple and lossy.
- **Summarise:** replace old turns with a short summary. Keeps the gist, costs a model call.
- **Keep the system prompt and the most recent turns intact**, and compress the middle.
- Doing nothing means an error, or silent truncation, at an unpredictable moment.

> ON THE EXAM
>
> Retrieved chunks compete with conversation history for the same window. Module 10's chunk size decision reappears here.

Converse or ConverseStream?

CONVERSE

One buffered response

- Returns the complete answer at once.
- Simpler to handle in code.
- Right for batch and backend work.
- The user waits for the whole generation.

CONVERSESTREAM

An ordered event stream

- messageStart, contentBlockDelta, messageStop.
- Text appears as it is generated.
- Right for anything a human is reading.
- Same request body as Converse.

THE TIEBREAKER

Streaming does not make generation faster. It makes the wait feel shorter, which for a user is most of the problem.

Time to first token, and time to finish

- **Time to first token** is how long before anything appears. Streaming exposes it.
- **Total generation time** scales with output length, because tokens are produced sequentially.
- A long answer is slow **by construction**. Shortening the answer is a latency fix.
- Smaller models, and distillation from module 11, reduce both.
- Some models offer latency-optimised inference for exactly this reason.

> ON THE EXAM
>
> If a scenario gives a latency budget, ask whether it applies to the first token or the last one. The answers differ.

Tool use, or function calling

- You describe tools in **toolConfig**: a name, a description, and an input schema.
- The model decides **when** to call one and with what arguments. It does not execute anything.
- **Your code executes the tool** and returns the result as another message. The model then answers.
- This is how a model gets live data: an order status, a stock level, a customer record.
- It also means the model can trigger actions, so what you expose is a security decision.

> ON THE EXAM
>
> Tool use is the alternative to fine-tuning that people miss. You cannot train facts that change hourly; you can look them up.

Prompt injection

- Untrusted text in a prompt can contain **instructions the model follows**.
- A retrieved document or a user message can say 'ignore previous instructions and…'.
- With **tool use**, that stops being a content problem and becomes an **actions** problem.
- Mitigations: least privilege on tools, confirmation for destructive actions, and Guardrails.
- Treat model output as untrusted input to whatever consumes it, exactly like user input.

> ON THE EXAM
>
> The question to ask of any tool you expose: what is the worst thing that happens if the model is persuaded to call it?

Making output machine-readable

- Ask for a specific schema in the prompt, and set **temperature near zero**.
- Use **stopSequences** so generation ends cleanly at the close of the structure.
- **Tool use is the more reliable route:** the input schema forces a shape the model must fill.
- Always validate the output anyway. A model can still return something that does not parse.
- Few-shot examples of the exact format help more than a longer description of it.

> ON THE EXAM
>
> Structured output has a schema-shaped solution. Describing the format in prose is the weakest of the available options.

Experimenting in the console

- The Bedrock **playground** lets you try prompts and parameters interactively, with no code.
- Comparison mode runs the same prompt against several models side by side.
- It is the fastest way to find out whether prompting will be enough, before anyone builds anything.
- Settle temperature, the system prompt and the model choice here, then write code once.
- Prompt Management from module 12 is where the result gets versioned.

> ON THE EXAM
>
> Most of the decisions in this module can be made in twenty minutes of interactive experimentation.

Customisation earns its cost only when **prompting has demonstrably failed**.

The customisation techniques, side by side

TECHNIQUE	IT FIXES
Prompt engineering	Unclear instructions, wrong format, missing framing
Tool use	Facts the model cannot know, and actions it must trigger
Retrieval	Knowledge the model lacks, that changes and must be cited
Fine-tuning	Behaviour, style and format, when examples exist
Continued pre-training	Unfamiliar domain vocabulary, from a large corpus
Distillation	Cost and latency, when a big model already works

ON THE EXAM

Six techniques, six distinct problems. Naming the problem precisely is what picks the technique.

Comparing models before you commit

- Build a small set of **representative prompts with known good answers**. Thirty to fifty is enough.
- Run the same set against two or three candidate models, with identical parameters.
- Compare on **quality, latency and cost per request together**. The cheapest adequate model usually wins.
- **Bedrock Evaluations** automates this, including LLM-as-a-judge and human review.
- Comparing on a handful of ad-hoc questions proves nothing, and is what usually happens.

> ON THE EXAM
>
> Because Converse makes models interchangeable, this comparison costs an afternoon. There is little excuse for skipping it.

When fine-tuning is genuinely worth it

- You have **tried few-shot prompting** and measured that it is not good enough.
- The behaviour you want is **consistent and stable**, not changing month to month.
- Prompts have grown long and expensive, and fine-tuning would **shorten them permanently**.
- You have several hundred good examples, and someone to maintain them.
- If any of those is untrue, the cheaper technique is probably still the right one.

> ON THE EXAM
>
> The economic argument is underrated: a fine-tuned model can drop a 2,000-token system prompt from every single request.

Real systems use several techniques at once

- A system prompt sets behaviour. **Retrieval** supplies documents. **Tools** fetch live data.
- They answer different questions, so combining them is normal rather than excessive.
- **Fine-tuning can sit underneath all of it**, handling format so the prompt can be shorter.
- Each addition is another thing to version, monitor and explain when it misbehaves.
- Exam questions usually ask for one technique. Read carefully for whether a pipeline is being described.

> ON THE EXAM
>
> A single assistant might use all four. The exam tests whether you can name which one solves the stated problem.

Custom Model Import

- Import model weights you trained elsewhere and serve them through Bedrock.
- So a model trained in SageMaker AI or outside AWS gets Bedrock's serving and tooling.
- Supported architectures are a defined list. Check before planning around it.
- **The Converse API does not support imported models.** Use the Invoke API instead.
- Named in the C02 additions, under deploying foundation models.

ON THE EXAM

That Converse limitation is a real design constraint: portable code written against Converse will not work with an imported model.

Three ways a model gets invoked

MODE	WHEN IT APPLIES
On-demand	Base models. Pay per token, no commitment, no capacity reserved
Provisioned Throughput	Guaranteed capacity, priced per model unit-hour
Custom model	Generally via Provisioned Throughput. On-demand for a growing subset

ON THE EXAM

Do not learn either 'custom models always need Provisioned Throughput' or the reverse. Both are wrong as universal rules.

Pinning a model version

- Model IDs carry a version, and providers ship new snapshot versions over time.
- A model that changes underneath you changes your outputs, with no deployment on your side.
- **Pin the exact model ID** in code, and upgrade deliberately after re-testing prompts.
- Re-run your evaluation set on the new version before switching. Prompts are version-sensitive.
- Models also move to Legacy and are eventually removed, from module 12. Plan the migration.

> ON THE EXAM
>
> Prompts are tuned against a specific model version. Upgrading the model without re-testing the prompt is an untested change.

Controlling which models can be used

- Model access is requested per model in the account, and governed per Region.
- **Access to some models cannot be removed** once granted, including Titan, Nova, DeepSeek-R1, Mistral and Llama.
- **Block them with an IAM policy on the model ID instead.** That is the documented approach.
- Useful for cost control as well as governance: deny the expensive models to most roles.
- Guardrails, in module 36, control what is said rather than which model says it.

> ON THE EXAM
>
> A governance question about which models a team may call is an IAM question, not a model-access question.

Logging model invocations

- **Model invocation logging** records requests and responses to S3 or CloudWatch Logs.
- It is the source of the **distillation prompts** from module 11, and of production evaluation data.
- **requestMetadata** lets you tag invocations with your own key-value pairs for filtering.
- Logs contain prompts and responses, so they inherit the sensitivity of both.
- It is **off by default**. Enable it before you need it, not after.

> ON THE EXAM
>
> Invocation logs are the only record of what a model was actually asked and what it actually said. Without them, incidents are unreconstructable.

What drives foundation model cost

DRIVER	EFFECT
Input tokens	Every prompt, every retrieved chunk, and the whole conversation history
Output tokens	Usually priced higher than input. maxTokens caps it
Model choice	Frontier models can cost many times a small model for the same task
Prompt caching	Reuses a repeated prefix. Up to 90 percent off cached tokens
Batch inference	Cheaper per token than on-demand, for work nobody waits on
Provisioned Throughput	Fixed hourly cost. Cheaper only above a volume threshold
Distillation	Large up-front cost, large ongoing saving. Needs volume to repay

ON THE EXAM

Conversation history is the cost driver teams miss, because it grows silently with every turn.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Output varies between identical requests	Temperature above zero. Lower it
Answers stop mid-sentence	maxTokens reached. It truncates without an error
topK setting appears to do nothing	It is not in inferenceConfig. Use additionalModelRequestFields
The assistant forgets earlier turns	Converse is stateless. The app must resend history
Cost grows through a long conversation	Full history resent each turn. Trim, summarise, or cache
Model invents facts about your business	No retrieval and no tools. It has nothing to look up
Portable code fails on an imported model	Converse does not support Custom Model Import

ON THE EXAM

Six of these seven are configuration or architecture, not model quality. That ratio is the point of the module.

Change the **cheapest thing that could fix it**, and measure before moving up.

A team uses a Bedrock foundation model to extract six named fields from supplier invoices and return them as JSON. The fields are extracted correctly, but the same invoice submitted twice sometimes yields different formatting, and occasionally a field is reworded.

Downstream systems parse the JSON strictly and fail on any variation. An engineer proposes fine-tuning the model on invoice examples.

What should be tried first?

A Fine-tune the model on several hundred invoice and JSON pairs

B Move to a larger, more capable foundation model

C Add a knowledge base containing past invoices for the model to retrieve from

D Set temperature near zero, specify the schema, and use tool use to enforce it

◆ PAUSE AND DECIDE

What should be tried first?

A. Fine-tune the model on several hundred invoice and JSON pairs

Would probably work, at the cost of a dataset, a training job and permanent maintenance, for a problem a free parameter change addresses.

B. Move to a larger, more capable foundation model

Extraction accuracy is already fine. A larger model at default temperature still samples, so the variation remains and costs more.

C. Add a knowledge base containing past invoices for the model to retrieve from

Retrieval supplies missing knowledge. Nothing is missing here: the invoice is in the prompt and the fields are found correctly.

D. Set temperature near zero, specify the schema, and use tool use to enforce it

The symptom is variation, not error. Near-zero temperature makes output near-deterministic, and a tool input schema forces the shape.

THE SIGNAL

Read the symptom precisely. 'Extracted correctly but worded differently each time' is variation, and variation is temperature.

A developer wants tighter control over token selection and adds a topK value of 40 to the inferenceConfig object of a Converse API call.

The call succeeds with no error and no warning, but output behaviour is unchanged. Adjusting the value to 5 and then to 200 makes no observable difference either.

What is happening?

A topK requires Provisioned Throughput to take effect

B topK is not in the common inferenceConfig set and must go in additionalModelRequestFields

C topK and topP conflict, so the model ignores both

D topK only applies to ConverseStream, not to Converse

◆ PAUSE AND DECIDE

What is happening?

A topK requires Provisioned Throughput to take effect

Throughput mode governs capacity and billing, not which inference parameters are honoured.

B topK is not in the common inferenceConfig set and must go in additionalModelRequestFields

inferenceConfig holds exactly four parameters: maxTokens, temperature, topP and stopSequences. Anything model-specific goes in the additional fields.

C topK and topP conflict, so the model ignores both

They can be set together on models that support both. And topP alone would still be having an effect, which it is not.

D topK only applies to ConverseStream, not to Converse

The two take the same request body. They differ in how the response is returned, not in which parameters are accepted.

THE SIGNAL

The tell is 'no error and no effect'. A parameter in the wrong place is ignored silently, which is why this is hard to debug.

A support chatbot built on the Converse API answers each question well, but users complain it has no memory. Ask a follow-up such as 'and how much does that cost?' and it responds as though the previous question never happened.

The team confirms the model supports multi-turn conversation and that no errors are being returned.

What is the cause?

A Converse is stateless, so the application must resend the full message history

B The conversation exceeded the model's context window and was silently truncated

C Prompt caching is discarding the earlier turns between requests

D The model needs fine-tuning on multi-turn conversation examples

◆ PAUSE AND DECIDE

What is the cause?

A Converse is stateless, so the application must resend the full message history

There is no server-side session. The caller owns the messages array and must resend every prior turn, or the model sees only the latest one.

B The conversation exceeded the model's context window and was silently truncated

That would affect long conversations, not the second turn. Here even the immediately preceding question is missing.

C Prompt caching is discarding the earlier turns between requests

Caching reuses a repeated prefix to reduce cost. It never removes content from a request that was sent.

D The model needs fine-tuning on multi-turn conversation examples

The model handles multi-turn conversation fine. It cannot use history it was never given, and no amount of training changes that.

THE SIGNAL

This is an application bug that presents as a model limitation. Statelessness is a documented property, not a fault.

> A logistics company wants a Bedrock assistant that answers questions like 'where is order 88213 now?'. Order status lives in a DynamoDB table and changes continuously throughout the day.
>
> A team proposes fine-tuning a model nightly on the current order table, so it knows the latest statuses.

How should you respond?

A Agree, but run continued pre-training instead, since the corpus is unlabelled

B Agree, but move to hourly fine-tuning so statuses are fresher

C Reject it. Define a tool the model can call to look up order status live

D Reject it. Build a knowledge base over nightly exports of the order table

◆ PAUSE AND DECIDE

How should you respond?

A Agree, but run continued pre-training instead, since the corpus is unlabelled

Swaps one training technique for another. Both bake a snapshot into weights, and the data changes continuously.

B Agree, but move to hourly fine-tuning so statuses are fresher

Twenty-four training jobs a day to chase data that changes every few minutes. It makes the wrong approach more expensive.

C Reject it. Define a tool the model can call to look up order status live

A single live query returns the current status at the moment it is asked. No staleness, no training, and it scales to any number of orders.

D Reject it. Build a knowledge base over nightly exports of the order table

Right to reject training, and it re-introduces staleness. Retrieval over a nightly export still answers with last night's status.

THE SIGNAL

Module 01 said knowledge problem, retrieve. This refines it: knowledge that changes by the minute needs a live lookup, not an index.

What to take from this module

- **inferenceConfig has four parameters.** topK goes in additionalModelRequestFields.
- **Converse is stateless.** The application resends history, and pays for it every turn.
- **Variation between identical requests is temperature.** Truncation is maxTokens.
- **Tool use for data that changes.** Retrieval for documents. Fine-tuning for behaviour.
- **Change the cheapest thing that could fix it**, and measure before moving up the ladder.

> ON THE EXAM
>
> Module 17 moves to measurement: how you tell whether a traditional model is any good.